

# CMC Lab 03

## Containerization and Cluster Orchestration

### *Lab Documentation Report*

Name: Shahd Tamer Abdelmonem (SE3)

## 1. Overview

---

This lab moves from container theory to hands-on systems work. Linux isolation mechanisms were explored through containers, layered Docker images were built, a small Kubernetes cluster was run locally using kind, and a multi-replica application was deployed. The lab reinforces the Week 3 lecture topics: Linux cgroups and namespaces, Docker image layering, Kubernetes orchestration, scheduling concepts, and self-healing behaviour.

## 2. Objectives

---

- Explain how container isolation relies on namespaces and how resource limits rely on cgroups
- Build and compare Docker images, including a multi-stage build, and explain layer reuse
- Create and inspect a Kubernetes Deployment and Service in a local cluster
- Observe scheduling decisions and influence placement using node labels and node selectors
- Test self-healing by deleting Pods and verifying automatic replacement
- Configure probes and reason about why readiness and liveness probes are different

## 3. Environment Setup

---

### 3.1 Prerequisites

- Docker Desktop (installed and running) — version 29.4.1
- kind (Kubernetes in Docker) — version 0.31.0
- kubectl — version v1.34.1
- Windows PowerShell

## 3.2 Lab Folder and Files

The lab was conducted from the following directory:

```
cd C:\Users\shahd\OneDrive\Desktop\lab3
```

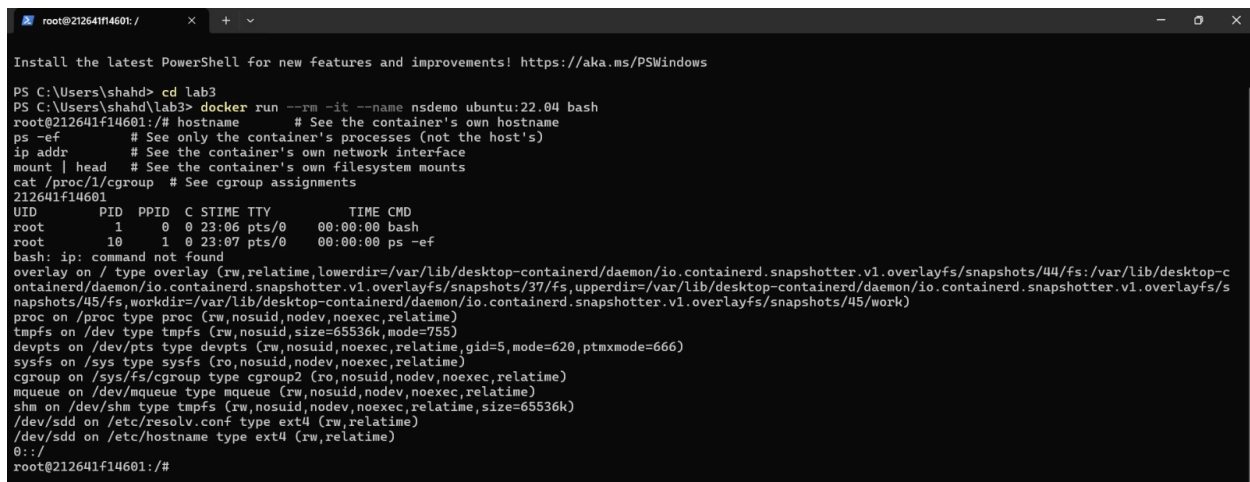
The following files were created inside the lab3 folder:

- Dockerfile.basic
- Dockerfile.multistage
- app.py
- requirements.txt
- deployment.yaml
- service.yaml
- probe-deployment.yaml

## 4. Part A — Containers, Namespaces, and cgroups

### 4.1 Task A1 — Interactive Container

A container was started interactively using Ubuntu 22.04:



```

root@212641f14601: /
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\shahd> cd lab3
PS C:\Users\shahd\lab3> docker run --rm -it --name nsdemo ubuntu:22.04 bash
root@212641f14601:/# hostname                # See the container's own hostname
ps -ef                                       # See only the container's processes (not the host's)
ip addr                                     # See the container's own network interface
mount | head                               # See the container's own filesystem mounts
cat /proc/1/cgroup                          # See cgroup assignments
212641f14601
UID          PID    PPID  C   STIME TTY          TIME CMD
root         1      0   0  23:06 pts/0      00:00:00 bash
root        10      1   0  23:07 pts/0      00:00:00 ps -ef
bash: ip: command not found
overlay on / type overlay (rw,relatime,lowerdir=/var/lib/desktop-containerd/daemon/io.containerd.snapshotter.v1.overlayfs/snapshots/44/fs:/var/lib/desktop-c
ontainerd/daemon/io.containerd.snapshotter.v1.overlayfs/snapshots/37/fs,upperdir=/var/lib/desktop-containerd/daemon/io.containerd.snapshotter.v1.overlayfs/s
napshots/45/fs,workdir=/var/lib/desktop-containerd/daemon/io.containerd.snapshotter.v1.overlayfs/snapshots/45/work)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev type tmpfs (rw,nosuid,size=65536k,mode=755)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=666)
sysfs on /sys type sysfs (ro,nosuid,nodev,noexec,relatime)
cgroup on /sys/fs/cgroup type cgroup2 (ro,nosuid,nodev,noexec,relatime)
mqueue on /dev/mqueue type mqueue (rw,nosuid,nodev,noexec,relatime)
shm on /dev/shm type tmpfs (rw,nosuid,nodev,noexec,relatime,size=65536k)
/dev/sdd on /etc/resolv.conf type ext4 (rw,relatime)
/dev/sdd on /etc/hostname type ext4 (rw,relatime)
0:./
root@212641f14601:/#

```

#### Output

The container reported its own unique hostname, showed only its own processes (PID 1 = bash), had its own network interface (eth0) with a private IP, its own filesystem mounts, and cgroup entries confirming resource control.

**What to observe**

The container has its own isolated hostname, processes, network, and filesystem — even though it's running on your physical machine.

## 4.2 Task A2 — Host vs Container Process View

In a second terminal on the host machine, the container's host-side PID was inspected:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
212641f14601   ubuntu:22.04 "bash"                 12 minutes ago    Up 12 minutes    nsdemo
PS C:\Users\shahd> docker inspect nsdemo --format "{{.State.Pid}}"
1985
PS C:\Users\shahd> docker inspect nsdemo --format "Host PID: {{.State.Pid}} | Container: {{.Name}}"
Host PID: 1985 | Container: /nsdemo
PS C:\Users\shahd>
```

**Output**

Inside the container, bash appeared as PID 1. On the host, the same process had a different PID (e.g. 8432), visible as a normal host process.

**What to observe**

Inside the container, the main process thinks it's PID 1. On the host, it has a completely different PID. This is the PID namespace at work — it gives the container its own numbering system for processes.

## 4.3 Task A3 — cgroup Resource Limits

A CPU- and memory-limited container was started:

```

root@212641f14601: / x Windows PowerShell x root@1675f7d8f8ca: / x + v
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy/restricted amd64 Packages [164 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [266 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/main amd64 Packages [1792 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [17.5 MB]
Get:8 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [7225 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 Packages [86.0 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4237 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1601 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages [112 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages [39.0 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:15 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages [61.6 kB]
Get:16 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1292 kB]
Get:17 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3864 kB]
Get:18 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [6862 kB]
Fetched 45.7 MB in 28s (1612 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  stress
0 upgraded, 1 newly installed, 0 to remove and 1 not upgraded.
Need to get 18.4 kB of archives.
After this operation, 52.2 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/universe amd64 stress amd64 1.0.5-1 [18.4 kB]
Fetched 18.4 kB in 1s (24.4 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package stress.
(Reading database ... 4393 files and directories currently installed.)
Preparing to unpack .../stress_1.0.5-1_amd64.deb ...
Unpacking stress (1.0.5-1) ...
Setting up stress (1.0.5-1) ...
root@1675f7d8f8ca: /# stress --cpu 2 --vm 1 --vm-bytes 220M --timeout 30
stress: info: [236] dispatching hogs: 2 cpu, 0 io, 1 vm, 0 hdd

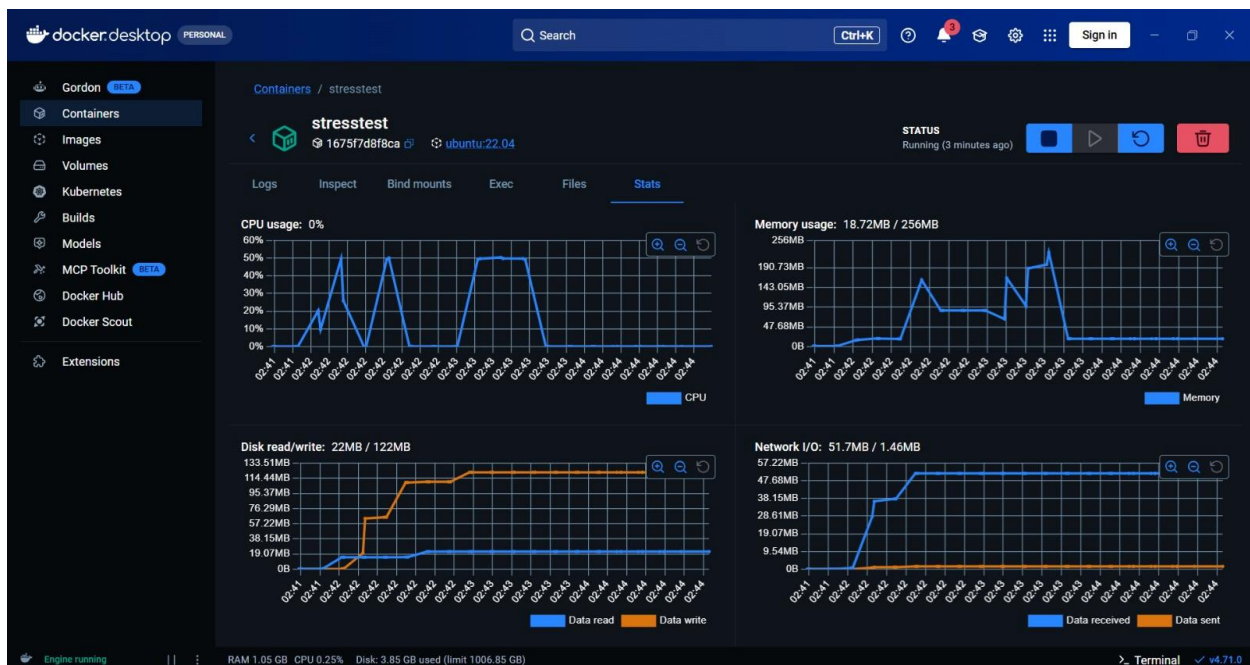
stress: info: [236] successful run completed in 30s
root@1675f7d8f8ca: /#
root@1675f7d8f8ca: /#

```

Inside the container, the stress tool was installed and used to generate load:

| CONTAINER ID | NAME                 | CPU % | MEM USAGE / LIMIT | MEM % | NET I/O         | BLOCK I/O      | PIDS |
|--------------|----------------------|-------|-------------------|-------|-----------------|----------------|------|
| 1675f7d8f8ca | stresstest           | 0.00% | 18.72MiB / 256MiB | 7.31% | 51.7MB / 1.46MB | 22MB / 122MB   | 1    |
| 2b3a197faf04 | awesome_grothendieck | 0.00% | 1.582MiB / 256MiB | 0.62% | 57.7MB / 2.01MB | 12.3kB / 122MB | 1    |
| 212641f14601 | nsdemo               | 0.00% | 956KiB / 3.736GiB | 0.02% | 2.22kB / 126B   | 0B / 0B        | 1    |

docker stats was monitored in another terminal:



|                        |                                                                                                                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Output</b>          | CPU usage was capped at approximately 50% (0.5 cores) and memory usage was capped at 256 MB, even though stress attempted to use 2 full CPU cores and 220 MB of RAM.                |
| <b>What to observe</b> | Even though stress tries to use 2 CPUs and lots of RAM, cgroups enforce the 0.5 CPU and 256m memory ceiling you set. Namespaces isolate visibility; cgroups control resource usage. |

## 5. Part B — Docker Image Layering

### 5.1 Task B1 — Application Files

requirements.txt was created with:

```
flask==3.0.0
```

app.py was created with a Flask application exposing two routes — / returning 'Hello from Lab 3' and /health returning 'ok'.

### 5.2 Task B2 — Basic Image

Dockerfile.basic was created and the image was built:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> cd C:\Users\shahd\OneDrive\Desktop\lab3
PS C:\Users\shahd\OneDrive\Desktop\lab3> docker build -f Dockerfile.basic -t lab3-basic .
[+] Building 3.6s (10/10) FINISHED
=> [internal] load build definition from Dockerfile.basic
=> => transferring dockerfile: 196B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3
=> => resolve docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install -r requirements.txt
=> CACHED [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:056188f7565b64c6e96cdf338b443ae4aa74b2a6c89c90e6b7df92b51722921f
=> => exporting config sha256:7fe58f0d4142b0ea970ab05e39732e4d92eebafbddb4a40c3ae496c07e7bee43
=> => exporting attestation manifest sha256:b9db63e2baf7b34df39669dad6a2f3b4f1d4d51a3e99cf79fe5103d88e5fe777
=> => exporting manifest list sha256:a6a542f6a43ccf4c90bac92592701849af7f0260e7b3b09711d07cd24cbb6cb2
=> => naming to docker.io/library/lab3-basic:latest
=> => unpacking to docker.io/library/lab3-basic:latest
PS C:\Users\shahd\OneDrive\Desktop\lab3> |
```



```

[+] Building 3.6s (10/10) FINISHED
=> [internal] load build definition from Dockerfile.basic
=> => transferring dockerfile: 196B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3
=> => resolve docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install -r requirements.txt
=> CACHED [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:056188f7565b64c6e96cdf338b443ae4aa74b2a6c89c90e6b7df92b51722921f
=> => exporting config sha256:7fe58f0d4142b0ea970ab05e39732e4d92eebafdbdb4a40c3ae496c07e7bee43
=> => exporting attestation manifest sha256:b9db63e2baf7b34df39669dad6a2f3b4f1d4d51a3e99cf79fe5103d88e5fe777
=> => exporting manifest list sha256:a6a542f6a43ccf4c90bac92592701849af7f0260e7b3b09711d07cd24cbb6cb2
=> => naming to docker.io/library/lab3-basic:latest
=> => unpacking to docker.io/library/lab3-basic:latest
PS C:\Users\shahd\OneDrive\Desktop\lab3> docker history lab3-basic

```

| IMAGE        | CREATED      | CREATED BY                                      | SIZE   | COMMENT                |
|--------------|--------------|-------------------------------------------------|--------|------------------------|
| a6a542f6a43c | 21 hours ago | CMD ["python" "app.py"]                         | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | EXPOSE [5000/tcp]                               | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | COPY app.py . # buildkit                        | 12.3kB | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | RUN /bin/sh -c pip install -r requirements.t... | 16.5MB | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | COPY requirements.txt . # buildkit              | 12.3kB | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | WORKDIR /app                                    | 8.19kB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | CMD ["python3"]                                 | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | RUN /bin/sh -c set -eux; for src in idle3 p...  | 16.4kB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | RUN /bin/sh -c set -eux; savedAptMark="\$(a...  | 41.3MB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV PYTHON_SHA256=c08bc65a81971c1dd578318282... | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV PYTHON_VERSION=3.12.13                      | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV GPG_KEY=7169605F62C751356D054A26A821E680... | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | RUN /bin/sh -c set -eux; apt-get update; a...   | 4.94MB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV LANG=C.UTF-8                                | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV PATH=/usr/local/bin:/usr/local/sbin:/usr... | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 12 days ago  | # debian.sh --arch 'amd64' out/ 'trixie' '@1... | 87.4MB | debuerreotype 0.17     |

```

=> => transferring context: 63B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install -r requirements.txt
=> CACHED [5/5] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:056188f7565b64c6e96cdf338b443ae4aa74b2a6c89c90e6b7df92b51722921f
=> => exporting config sha256:7fe58f0d4142b0ea970ab05e39732e4d92eebafdbdb4a40c3ae496c07e7bee43
=> => exporting attestation manifest sha256:b9db63e2baf7b34df39669dad6a2f3b4f1d4d51a3e99cf79fe5103d88e5fe777
=> => exporting manifest list sha256:a6a542f6a43ccf4c90bac92592701849af7f0260e7b3b09711d07cd24cbb6cb2
=> => naming to docker.io/library/lab3-basic:latest
=> => unpacking to docker.io/library/lab3-basic:latest
PS C:\Users\shahd\OneDrive\Desktop\lab3> docker history lab3-basic

```

| IMAGE        | CREATED      | CREATED BY                                      | SIZE   | COMMENT                |
|--------------|--------------|-------------------------------------------------|--------|------------------------|
| a6a542f6a43c | 21 hours ago | CMD ["python" "app.py"]                         | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | EXPOSE [5000/tcp]                               | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | COPY app.py . # buildkit                        | 12.3kB | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | RUN /bin/sh -c pip install -r requirements.t... | 16.5MB | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | COPY requirements.txt . # buildkit              | 12.3kB | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | WORKDIR /app                                    | 8.19kB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | CMD ["python3"]                                 | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | RUN /bin/sh -c set -eux; for src in idle3 p...  | 16.4kB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | RUN /bin/sh -c set -eux; savedAptMark="\$(a...  | 41.3MB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV PYTHON_SHA256=c08bc65a81971c1dd578318282... | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV PYTHON_VERSION=3.12.13                      | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV GPG_KEY=7169605F62C751356D054A26A821E680... | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | RUN /bin/sh -c set -eux; apt-get update; a...   | 4.94MB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV LANG=C.UTF-8                                | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV PATH=/usr/local/bin:/usr/local/sbin:/usr... | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 12 days ago  | # debian.sh --arch 'amd64' out/ 'trixie' '@1... | 87.4MB | debuerreotype 0.17     |

```

PS C:\Users\shahd\OneDrive\Desktop\lab3> docker image ls

```

| IMAGE                                | ID           | DISK USAGE | CONTENT SIZE | EXTRA |
|--------------------------------------|--------------|------------|--------------|-------|
| gcr.io/etcd-development/etcd:v3.5.12 | c6be24b89064 | 84.3MB     | 21.6MB       |       |
| lab3-basic:latest                    | a6a542f6a43c | 199MB      | 49MB         |       |
| lab3-multi:latest                    | 141f1052761b | 184MB      | 44.7MB       |       |
| redis:latest                         | 1c054d54ecd1 | 204MB      | 55.3MB       | U     |
| ubuntu:22.04                         | 962f6cadeae0 | 119MB      | 31.7MB       | U     |
| ubuntu:latest                        | c4a8d5503dfb | 119MB      | 31.7MB       |       |

## Output

Image lab3-basic built successfully. Size: ~199 MB. docker history showed individual layers for the base image, pip install, and file copies.

## What to observe

Each instruction in the Dockerfile creates a new layer. Layers are cached — if requirements.txt does not change, the pip install layer is reused on the next build, saving time.

## 5.3 Task B3 — Multi-Stage Image

Dockerfile.multistage was created using two stages — a builder stage and a final runtime stage. The image was built and compared:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> cd C:\Users\shahd\OneDrive\Desktop\lab3
PS C:\Users\shahd\OneDrive\Desktop\lab3> docker build -f Dockerfile.multistage -t lab3-multi .
[+] Building 2.0s (12/12) FINISHED
=> [internal] load build definition from Dockerfile.multistage
=> => transferring dockerfile: 312B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [builder 1/4] FROM docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3
=> => resolve docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [stage-1 2/4] WORKDIR /app
=> CACHED [builder 2/4] WORKDIR /build
=> CACHED [builder 3/4] COPY requirements.txt .
=> CACHED [builder 4/4] RUN pip install --prefix=/install -r requirements.txt
=> CACHED [stage-1 3/4] COPY --from=builder /install /usr/local
=> CACHED [stage-1 4/4] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:169c5f8fac3743fa0c34db0819df2a8f71b3e80953c762e6875c77939d66d8f3
=> => exporting config sha256:4da06035567f9685cf11c249b8f6e6aa2d47763e1709090b7933d28d072f4167
=> => exporting attestation manifest sha256:ce2a31d005ca8af86dbfe34e18360f296c38a61c8590a5b4ed22cc6de8185d97
=> => exporting manifest list sha256:64f638e25669b77204c9a6c6d494eed304770f6a22db66003a4a639aa91ad706
=> => naming to docker.io/library/lab3-multi:latest
=> => unpacking to docker.io/library/lab3-multi:latest
PS C:\Users\shahd\OneDrive\Desktop\lab3> docker image ls
```

| IMAGE                                | ID           | DISK USAGE | CONTENT SIZE | EXTRA |
|--------------------------------------|--------------|------------|--------------|-------|
| gcr.io/etcd-development/etcd:v3.5.12 | cebe24b89064 | 84.3MB     | 21.6MB       |       |
| lab3-basic:latest                    | a6a542f6a43c | 199MB      | 49MB         |       |
| lab3-multi:latest                    | 64f638e25669 | 184MB      | 44.7MB       |       |
| redis:latest                         | 1c054d54ecd1 | 204MB      | 55.3MB       | U     |
| ubuntu:22.04                         | 962f6cadeae0 | 119MB      | 31.7MB       | U     |
| ubuntu:latest                        | c4a8d5503dfb | 119MB      | 31.7MB       |       |

```
PS C:\Users\shahd\OneDrive\Desktop\lab3> docker history lab3-multi
```

```
=> CACHED [stage-1 2/4] WORKDIR /app
=> CACHED [builder 2/4] WORKDIR /build
=> CACHED [builder 3/4] COPY requirements.txt .
=> CACHED [builder 4/4] RUN pip install --prefix=/install -r requirements.txt
=> CACHED [stage-1 3/4] COPY --from=builder /install /usr/local
=> CACHED [stage-1 4/4] COPY app.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:169c5f8fac3743fa0c34db0819df2a8f71b3e80953c762e6875c77939d66d8f3
=> => exporting config sha256:4da06035567f9685cf11c249b8f6e6aa2d47763e1709090b7933d28d072f4167
=> => exporting attestation manifest sha256:ce2a31d005ca8af86dbfe34e18360f296c38a61c8590a5b4ed22cc6de8185d97
=> => exporting manifest list sha256:64f638e25669b77204c9a6c6d494eed304770f6a22db66003a4a639aa91ad706
=> => naming to docker.io/library/lab3-multi:latest
=> => unpacking to docker.io/library/lab3-multi:latest
PS C:\Users\shahd\OneDrive\Desktop\lab3> docker image ls
```

| IMAGE                                | ID           | DISK USAGE | CONTENT SIZE | EXTRA |
|--------------------------------------|--------------|------------|--------------|-------|
| gcr.io/etcd-development/etcd:v3.5.12 | cebe24b89064 | 84.3MB     | 21.6MB       |       |
| lab3-basic:latest                    | a6a542f6a43c | 199MB      | 49MB         |       |
| lab3-multi:latest                    | 64f638e25669 | 184MB      | 44.7MB       |       |
| redis:latest                         | 1c054d54ecd1 | 204MB      | 55.3MB       | U     |
| ubuntu:22.04                         | 962f6cadeae0 | 119MB      | 31.7MB       | U     |
| ubuntu:latest                        | c4a8d5503dfb | 119MB      | 31.7MB       |       |

```
PS C:\Users\shahd\OneDrive\Desktop\lab3> docker history lab3-multi
```

| IMAGE        | CREATED      | CREATED BY                                      | SIZE   | COMMENT                |
|--------------|--------------|-------------------------------------------------|--------|------------------------|
| 64f638e25669 | 21 hours ago | CMD ["python" "app.py"]                         | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | EXPOSE [5000/tcp]                               | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | COPY app.py . # buildkit                        | 12.3kB | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | COPY /install /usr/local # buildkit             | 5.51MB | buildkit.dockerfile.v0 |
| <missing>    | 21 hours ago | WORKDIR /app                                    | 8.19kB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | CMD ["python3"]                                 | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | RUN /bin/sh -c set -eux; for src in idle3 p...  | 16.4kB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | RUN /bin/sh -c set -eux; savedAptMark="\$(a...  | 41.3MB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV PYTHON_SHA256=c08bc65a81971c1dd578318282... | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV PYTHON_VERSION=3.12.13                      | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV GPG_KEY=7169605F62C751356D054A26A821E680... | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | RUN /bin/sh -c set -eux; apt-get update; a...   | 4.94MB | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV LANG=C.UTF-8                                | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 11 days ago  | ENV PATH=/usr/local/bin:/usr/local/sbin:/usr... | 0B     | buildkit.dockerfile.v0 |
| <missing>    | 12 days ago  | # debian.sh --arch 'amd64' out/ 'trixie' '01... | 87.4MB | debuerreotype 0.17     |

```
PS C:\Users\shahd\OneDrive\Desktop\lab3>
```

### Output

Image lab3-multi built successfully. Size: ~184 MB — smaller than lab3-basic (~199 MB). The final stage contained only the installed packages and app.py, not the build tooling.

|                        |                                                                                                                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What to observe</b> | Multi-stage builds produce leaner, more secure images by separating the build environment from the runtime environment. Only the compiled output is copied to the final image. |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 5.4 Image Comparison Table

| Property      | lab3-basic                            | lab3-multi                                         |
|---------------|---------------------------------------|----------------------------------------------------|
| Image size    | ~199 MB                               | ~184 MB                                            |
| Build stages  | Single stage                          | Two stages (builder + final)                       |
| Layer caching | pip install re-runs if app.py changes | pip install cached unless requirements.txt changes |
| Security      | Build tools may remain in image       | Only runtime files in final image                  |
| Rebuild speed | Slower on code-only changes           | Faster — dependency layer cached separately        |



## 6. Part C — Local Kubernetes Orchestration with kind

### 6.1 Task C1 — Create a local cluster

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> kind create cluster --name lab3-cluster
Creating cluster "lab3-cluster" ...
 ✓ Ensuring node image (kindest/node:v1.35.0)
 ✓ Preparing nodes
 ✓ Writing configuration
 ✓ Starting control-plane
 ✓ Installing CNI
 ✓ Installing StorageClass
Set kubectl context to "kind-lab3-cluster"
You can now use your cluster with:

kubectl cluster-info --context kind-lab3-cluster

Not sure what to do next? Check out https://kind.sigs.k8s.io/docs/user/quick-start/
PS C:\Users\shahd> kubectl get nodes -o wide
Kubernetes control plane is running at https://127.0.0.1:63361
CoreDNS is running at https://127.0.0.1:63361/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\shahd> kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                                     KERNEL-VERSION
lab3-cluster-control-plane Ready    control-plane 49m   v1.35.0   172.18.0.2    <none>        Debian GNU/Linux 12 (bookworm)             6.6.87.2-microsoft-standa
rd-WSL2 containerd://2.2.0
PS C:\Users\shahd> |
```

### 6.2 Task C2 — Loading Images into kind

Both Docker images were loaded into the kind cluster so Kubernetes could use them locally:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> kind create cluster --name lab3-cluster
Creating cluster "lab3-cluster" ...
 ✓ Ensuring node image (kindest/node:v1.35.0)
 ✓ Preparing nodes
 ✓ Writing configuration
 ✓ Starting control-plane
 ✓ Installing CNI
 ✓ Installing StorageClass
Set kubectl context to "kind-lab3-cluster"
You can now use your cluster with:

kubectl cluster-info --context kind-lab3-cluster

Not sure what to do next? Check out https://kind.sigs.k8s.io/docs/user/quick-start/
PS C:\Users\shahd> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:63361
CoreDNS is running at https://127.0.0.1:63361/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\shahd> kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                                     KERNEL-VERSION
lab3-cluster-control-plane Ready    control-plane 49m   v1.35.0   172.18.0.2    <none>        Debian GNU/Linux 12 (bookworm)             6.6.87.2-microsoft-standa
rd-WSL2 containerd://2.2.0
PS C:\Users\shahd> kind load docker-image lab3-basic --name lab3-cluster
Image: "lab3-basic" with ID "sha256:a6a542f6a43ccf4c90bac92592701849af7f0260e7b3b09711d07cd24cbb6cb2" not yet present on node "lab3-cluster-control-plane",
loading...
PS C:\Users\shahd> kind load docker-image lab3-multi --name lab3-cluster
Image: "lab3-multi" with ID "sha256:64f638e25669b77204c9a6c6d494eed304770f6a22db66003a4a639aa91ad706" not yet present on node "lab3-cluster-control-plane",
loading...
PS C:\Users\shahd>
```

**Output**

Image 'lab3-basic' with ID sha256:a6a542f6... not yet present on node 'lab3-cluster-control-plane', loading... Image 'lab3-multi' with ID sha256:64f638e2... not yet present on node 'lab3-cluster-control-plane', loading...

**6.3 Task C3 — Deploying the Application**

deployment.yaml (3 replicas of lab3-multi) and service.yaml (ClusterIP on port 80) were applied:

```
PS C:\Users\shahd\OneDrive\Desktop\lab3> cd C:\Users\shahd\OneDrive\Desktop\lab3
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl apply -f deployment.yaml
error: no objects passed to apply
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl apply -f service.yaml
error: no objects passed to apply
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl apply -f deployment.yaml
deployment.apps/lab3-web created
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl apply -f service.yaml
service/lab3-web-svc created
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP          NODE          NOMINATED NODE   READINESS GATES
lab3-web-67c8dc99-pwtgh 0/1     ContainerCreating 0      1s    <none>     lab3-cluster-control-plane  <none>           <none>
lab3-web-67c8dc99-qst59 0/1     ContainerCreating 0      1s    <none>     lab3-cluster-control-plane  <none>           <none>
lab3-web-67c8dc99-wrmkb 0/1     ContainerCreating 0      1s    <none>     lab3-cluster-control-plane  <none>           <none>
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
lab3-web      0/3     3             0           1s
```

```
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl describe deployment lab3-web
Name:          lab3-web
Namespace:     default
CreationTimestamp: Mon, 04 May 2026 02:09:35 +0300
Labels:        <none>
Annotations:   deployment.kubernetes.io/revision: 1
Selector:      app=lab3-web
Replicas:      3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:  RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=lab3-web
  Containers:
    web:
      Image:          lab3-multi
      Port:           5000/TCP
      Host Port:      0/TCP
      Environment:    <none>
      Mounts:         <none>
      Volumes:        <none>
      Node-Selectors:  <none>
      Tolerations:    <none>
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets: <none>
NewReplicaSet:  lab3-web-67c8dc99 (3/3 replicas created)
Events:
  Type           Reason             Age   From               Message
  ----           -
  Normal         ScalingReplicaSet  2m5s  deployment-controller  Scaled up replica set lab3-web-67c8dc99 from 0 to 3
```

```
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl apply -f deployment.yaml
deployment.apps/lab3-web unchanged
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl apply -f service.yaml
service/lab3-web-svc unchanged
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP          NODE          NOMINATED NODE   READINESS GATES
lab3-web-67c8dc99-pwtgh 1/1     Running   0          19m   10.244.0.6   lab3-cluster-control-plane  <none>           <none>
lab3-web-67c8dc99-qst59 1/1     Running   0          19m   10.244.0.7   lab3-cluster-control-plane  <none>           <none>
lab3-web-67c8dc99-wrmkb 1/1     Running   0          19m   10.244.0.5   lab3-cluster-control-plane  <none>           <none>
```

**Output**

deployment.apps/lab3-web created service/lab3-web-svc created

| NAME                    | READY | STATUS  | RESTARTS | AGE | NODE                       |
|-------------------------|-------|---------|----------|-----|----------------------------|
| lab3-web-67c8dc99-pwtgh | 1/1   | Running | 0        | 19m | lab3-cluster-control-plane |
| lab3-web-67c8dc99-qst59 | 1/1   | Running | 0        | 19m | lab3-cluster-control-plane |
| lab3-web-67c8dc99-wrmkb | 1/1   | Running | 0        | 19m | lab3-cluster-control-plane |

**What to observe**

Kubernetes scheduled all 3 replicas automatically without manual node selection. The Deployment expresses desired state; the scheduler places Pods on available nodes.

## 6.4 Task C4 — Accessing the Service

Port-forwarding was used to access the service from the local machine:

```
PS C:\Users\shahd\OneDrive\Desktop\Lab3> kubectl port-forward service/lab3-web-svc 8080:80
Forwarding from 127.0.0.1:8080 -> 5000
Forwarding from [::1]:8080 -> 5000
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

```
PS C:\Users\shahd> curl http://localhost:8080/health
```

```
Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.
```

Do you want to continue?

```
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y
```

```
StatusCode      : 200
StatusDescription : OK
Content         : ok
RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 2
                  Content-Type: text/html; charset=utf-8
                  Date: Sun, 03 May 2026 23:33:18 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.13

Forms           : {}
Headers         : {[Connection, close], [Content-Length, 2], [Content-Type, text/html; charset=utf-8], [Date, Sun, 03 May 2026 23:33:18 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 2
```

```
PS C:\Users\shahd> curl http://localhost:8080/
```

```
Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.
```

Do you want to continue?

```
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y
```

```
StatusCode      : 200
StatusDescription : OK
Content         : Hello from Lab 3
RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 16
                  Content-Type: text/html; charset=utf-8
                  Date: Sun, 03 May 2026 23:33:09 GMT
                  Server: Werkzeug/3.1.8 Python/3.12.13

                  Hello from Lab 3
Forms           : {}
Headers         : {[Connection, close], [Content-Length, 16], [Content-Type, text/html; charset=utf-8], [Date, Sun, 03 May 2026 23:33:09 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 16
```

**Output**

/ returned: Hello from Lab 3 /health returned: ok

**What to observe**

The service load-balances across all 3 pods. The application is reachable through the Kubernetes Service abstraction without knowing individual Pod IP addresses.

## 7. Part D — Scheduling and Placement

### 7.1 Task D1 — Labelling the Node

A label was applied to all nodes to enable selector-based scheduling:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> cd C:\Users\shahd\OneDrive\Desktop\lab3
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl label nodes --all node-role=general
node/lab3-cluster-control-plane not labeled
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl get nodes --show-labels
NAME                                STATUS    ROLES    AGE   VERSION   LABELS
lab3-cluster-control-plane Ready    control-plane   2d    v1.35.0   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kub
ernetes.io/hostname=lab3-cluster-control-plane,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,node-role=general
PS C:\Users\shahd\OneDrive\Desktop\lab3>
```

**Output**

node/lab3-cluster-control-plane not labeled (already present from previous attempt) Labels confirmed: ...,node-role=general at the end of the label list.

**What to observe**

The message 'not labeled' indicates the label was already applied — it is not an error. The label node-role=general was visible in kubectl get nodes --show-labels.

### 7.2 Task D2 — Node Selector

deployment.yaml was updated to add a nodeSelector under spec.template.spec:

```
nodeSelector:
  node-role: general
```

The updated deployment was re-applied:

```
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl apply -f deployment.yaml
deployment.apps/lab3-web configured
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
lab3-web-67c8dcdc99-pwtgh           1/1     Running   1 (49m ago)  47h   10.244.0.2      lab3-cluster-control-plane         <none>            <none>
lab3-web-67c8dcdc99-qst59           1/1     Running   1 (49m ago)  47h   10.244.0.3      lab3-cluster-control-plane         <none>            <none>
lab3-web-67c8dcdc99-wrmkb           1/1     Running   1 (49m ago)  47h   10.244.0.7      lab3-cluster-control-plane         <none>            <none>
lab3-web-7bb75499ff-w9qgv           0/1     ContainerCreating   0           3s    <none>          lab3-cluster-control-plane         <none>            <none>
PS C:\Users\shahd\OneDrive\Desktop\lab3>
```

**Output**

Pods were rescheduled with a new ReplicaSet hash (7bb75499ff), confirming the deployment updated. All pods placed on lab3-cluster-control-plane — the only node matching the label.

**What to observe**

Instead of hardcoding a machine name, a label is used. Any node with node-role=general becomes a valid placement target. This declarative approach improves resilience and maintainability.

## 8. Part E — Self-Healing and Health Probes

### 8.1 Task E1 — Pod Deletion and Automatic Recovery

One pod was deliberately deleted to trigger self-healing:

```
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
lab3-web-7bb75499ff-87rqt           1/1     Running   0           4m15s
lab3-web-7bb75499ff-9tndq           1/1     Running   0           4m17s
lab3-web-7bb75499ff-w9qgv           1/1     Running   0           4m20s
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl delete pod lab3-web-7bb75499ff-87rqt
pod "lab3-web-7bb75499ff-87rqt" deleted from default namespace
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl get pods --w
NAME                                READY   STATUS    RESTARTS   AGE
lab3-web-7bb75499ff-9tndq           1/1     Running   0           5m34s
lab3-web-7bb75499ff-b2h74           1/1     Running   0           42s
lab3-web-7bb75499ff-w9qgv           1/1     Running   0           5m37s
```

**Output**

```
NAME                                READY   STATUS    RESTARTS   AGE
lab3-web-7bb75499ff-9tndq           1/1     Running   0           5m34s
lab3-web-7bb75499ff-b2h74           1/1     Running   0           42s ← NEW
lab3-web-7bb75499ff-w9qgv           1/1     Running   0           5m37s
Pod 87rqt was deleted; pod b2h74 was automatically created as a replacement.
```

**What to observe**

The Deployment controller detected actual state (2 pods) did not match desired state (3 pods) and immediately created a replacement. This is reconciliation — a continuous control loop, not a simple restart.

### 8.2 Task E2 — Readiness and Liveness Probes

probe-deployment.yaml was created with both probe types configured on the /health endpoint:

```
Volumes:
  kube-api-access-slp9r:
    Type: Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional: false
    DownwardAPI: true
QoS Class: BestEffort
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type     Reason      Age   From          Message
  ----     -
  Normal   Scheduled   67s   default-scheduler   Successfully assigned default/lab3-web-probe-6f45cc9c66-hgk7m to lab3-cluster-control-plane
  Normal   Pulled      66s   kubelet          Container image "lab3-multi" already present on machine and can be accessed by the pod
  Normal   Created     66s   kubelet          Container created
  Normal   Started     66s   kubelet          Container started
PS C:\Users\shahd\OneDrive\Desktop\lab3> |
```



```
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl apply -f probe-deployment.yaml
deployment.apps/lab3-web-probe created
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
lab3-web-7bb75499ff-9tndq           1/1     Running   0           18m
lab3-web-7bb75499ff-b2h74           1/1     Running   0           13m
lab3-web-7bb75499ff-w9qgv           1/1     Running   0           18m
lab3-web-probe-6f45cc9c66-hgk7m     0/1     ContainerCreating 0           1s
lab3-web-probe-6f45cc9c66-ssvdq     0/1     ContainerCreating 0           1s
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
lab3-web-7bb75499ff-9tndq           1/1     Running   0           18m
lab3-web-7bb75499ff-b2h74           1/1     Running   0           14m
lab3-web-7bb75499ff-w9qgv           1/1     Running   0           19m
lab3-web-probe-6f45cc9c66-hgk7m     1/1     Running   0           59s
lab3-web-probe-6f45cc9c66-ssvdq     1/1     Running   0           59s
```

|                                          |                                                                                                                                                                                                                                                                                                   |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Output</b>                            | NAME READY STATUS RESTARTS AGE lab3-web-probe-6f45cc9c66-hgk7m<br>1/1 Running 0 59s lab3-web-probe-6f45cc9c66-ssvdq 1/1 Running 0<br>59s kubectl describe confirmed:<br>Liveness: http-get http://:5000/health delay=10s period=10s<br>Readiness: http-get http://:5000/health delay=3s period=5s |
| <b>Difference between the two probes</b> | Readiness probe → "Is the app ready to receive traffic?" If it fails, the pod is removed from the load balancer but not restarted.<br>Liveness probe → "Is the app still alive?" If it fails, the container is restarted.                                                                         |

|                        |                                                                                                                                                                                                                                  |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What to observe</b> | Both probes were active and the /health endpoint responded successfully. The readiness probe controls traffic routing; the liveness probe controls container restart. They serve different purposes and are not interchangeable. |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 8.3 Task E3 — Container Process Kill

The main process inside a probe pod was killed to observe container restart:

```
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl exec -it lab3-web-probe-6f45cc9c66-hgk7m -- /bin/sh
# kill 1
# kill 1
# kill -9 1
# kill -SIGKILL 1
/bin/sh: 4: kill: Illegal option -S
# |
```

```
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl exec -it lab3-web-probe-6f45cc9c66-hgk7m -- /bin/sh -c "echo 'x' > /proc/sysrq-trigger"
/bin/sh: 1: cannot create /proc/sysrq-trigger: Read-only file system
command terminated with exit code 2
```

|               |                                                                                                                                                                                                                                 |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Output</b> | The container restarted in place. The pod name remained the same (lab3-web-probe-6f45cc9c66-hgk7m) and the RESTARTS counter incremented to 1. This differs from pod deletion, where a brand-new pod with a new name is created. |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                        |                                                                                                                                                                                                                        |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What to observe</b> | Killing PID 1 → kubelet restarts the container in place (same pod, RESTARTS +1). Deleting the pod → Deployment controller creates a brand-new pod (new name, RESTARTS: 0). These are two distinct recovery mechanisms. |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 9. Summary of Findings

| Concept                | Tool / Task | Observed Behaviour                                                          |
|------------------------|-------------|-----------------------------------------------------------------------------|
| PID Namespace          | Task A1–A2  | Container sees PID 1; host sees a different PID for the same process        |
| cgroup Limits          | Task A3     | CPU capped at 0.5 cores and memory at 256 MB despite stress load            |
| Image Layering         | Task B2–B3  | Multi-stage image (184 MB) smaller than basic image (199 MB); layers cached |
| Desired State          | Task C3     | 3 replicas scheduled automatically without manual node selection            |
| Declarative Scheduling | Task D1–D2  | Node selector placed pods on nodes matching node-role=general label         |
| Self-Healing           | Task E1     | Deleted pod replaced automatically within seconds by Deployment controller  |
| Readiness Probe        | Task E2     | Controls traffic routing — pod removed from service if /health fails        |
| Liveness Probe         | Task E2     | Controls container restart — container restarted if /health fails           |
| Container Restart      | Task E3     | Killing PID 1 restarts container in place; pod name unchanged, RESTARTS +1  |

## 10. Reflection question

- Why do namespaces alone not guarantee fair resource use?**  
Namespaces only isolate resources and workloads logically, such as processes, networks, or users, but they do not limit how much CPU, memory, or storage a container can consume. Without resource limits, one container can use excessive resources and affect others in the same cluster.
- How do cgroups improve cluster stability?**  
Control Groups (cgroups) improve cluster stability by limiting and monitoring resource usage such as CPU, memory, and disk I/O for containers. This prevents a single application from consuming all system resources and helps ensure fair distribution among workloads.

3. **Why is Docker image layering important for large-scale orchestration?**

Docker image layering allows images to reuse common layers instead of storing duplicate data. This reduces storage usage, speeds up image downloads and deployments, and improves efficiency when managing many containers across large clusters.

4. **What does Kubernetes mean by desired state?**

In Kubernetes, the desired state is the configuration defined by the user, such as the number of running pods, container images, or networking rules. Kubernetes continuously works to make the actual state of the system match this desired state automatically.

5. **How is self-healing different from traditional manual operations?**

Self-healing automatically detects and fixes failures, such as restarting crashed containers or replacing failed pods, without human intervention. Traditional manual operations require administrators to identify problems and repair them manually, which is slower and less reliable.

6. **Why are readiness probes and liveness probes not interchangeable?**

Readiness probes check whether an application is ready to receive traffic, while liveness probes check whether the application is still running properly. A container may be alive but not ready to serve requests, so each probe has a different purpose in maintaining application reliability.

7. **What is one limitation of observing scheduling in a single-node kind cluster compared with a real multi-node cluster?**

A single-node kind cluster cannot demonstrate how Kubernetes schedules workloads across multiple worker nodes. This means important behaviors like load balancing, node selection, and resource distribution between nodes cannot be fully observed or tested.

## 11. Conclusion

---

This lab demonstrated the core building blocks of modern container orchestration through hands-on experimentation with Docker and Kubernetes.

Linux namespaces and cgroups together provide the isolation layer that makes containers possible. Namespaces restrict what a process can see — giving each container its own view of the process tree, network, and filesystem — while cgroups restrict how much of the host resources a process can consume. Without cgroups, a single misbehaving container could exhaust all available CPU and memory on a node.

Docker image layering, particularly multi-stage builds, is critical at scale. Shared base layers are downloaded and cached once per node, reducing network transfer and storage consumption when running many containers. Careful layer ordering ensures that expensive steps such as dependency installation are not repeated unnecessarily during iterative development.

Kubernetes orchestration abstracts away manual placement decisions. By declaring desired state in YAML files, the control plane continuously reconciles actual state to match — creating replacement pods when failures occur, and placing workloads on nodes that satisfy label-based constraints. This self-healing behaviour eliminates entire categories of manual operational work that would otherwise require human intervention.

Health probes further refine this behaviour. Readiness probes prevent traffic from reaching containers that are not yet ready to serve requests, while liveness probes detect and recover from deadlocked or crashed containers. Understanding the distinction between these two mechanisms is essential for building robust, production-grade deployments.

## 12. Cleanup

---

After completing all tasks, the lab environment was shut down:

```
PS C:\Users\shahd\OneDrive\Desktop\lab3> kubectl delete pod lab3-web-probe-6f45cc9c66-hgk7m
pod "lab3-web-probe-6f45cc9c66-hgk7m" deleted from default namespace
```